

F42: Fortran coding guides

Asis Hallab, “el Bobito”

June 12, 2025

1 Memory Passing Behavior in Fortran (F42 Standard for Tensor Omics)

1.1 General Principles

In Fortran (standards 77–2018), arguments to subroutines and functions are, by default, passed **by reference** rather than by copying. This applies to scalars, arrays, and derived types.

Thus:

- Scalars (e.g., `integer`, `real`) are passed by reference.
- Arrays (e.g., `integer`, `dimension(:)`, `real`, `dimension(:, :)`) are passed by reference.
- Derived types are passed by reference.

1.2 When Copies Are Made

Despite the default pass-by-reference behavior, hidden copies may be generated by the compiler in the following situations:

- **Passing expressions:** e.g., call `process(a+b)` requires evaluation and temporary storage.
- **Passing strided array sections:** e.g., `array(1:10:2)` is non-contiguous and may trigger copying.
- **Passing non-contiguous slices:** particularly relevant for rows in a 2D array.

1.3 Tensor Omics DataTables: Column and Row Passing

Tensor Omics DataTables store typed data in 2D Fortran arrays of the form:

- `real_array(n_rows, n_real_columns)`
- `int_array(n_rows, n_int_columns)`
- `char_array(n_rows, n_char_columns)`

Accessing Columns:

- Example: `real_array(:, j)`
- Behavior: Passed by reference, **no copy made**.
- Reason: Slicing across the first index in column-major layout yields contiguous memory.

Accessing Rows:

- Example: `real_array(i, :)`
- Behavior: Typically triggers a hidden copy.
- Reason: Slicing across the second index accesses non-contiguous memory addresses, requiring packing into a contiguous temporary.

1.4 Summary Tables

1.4.1 General Fortran Behavior

Action	Example	Copy Made?
Pass full array	<code>array</code>	No
Pass contiguous slice	<code>array(:, j)</code>	No
Pass strided slice	<code>array(1:10:2, j)</code>	Yes
Pass expression	<code>array(:, j) * 2.0</code>	Yes

1.4.2 Tensor Omics DataTables Specifics

Access Type	Example	Copy Made?
Access full column	<code>real_array(:, j)</code>	No
Access full row	<code>real_array(i, :)</code>	Yes

1.5 Best Practice Recommendation for Tensor Omics

- Always design core functions to work with columns if possible.
- Avoid passing strided slices or expressions directly to subroutines.
- Treat rows carefully: copy manually if absolutely needed, otherwise prefer per-element processing.

1.6 Conclusion

Efficient memory handling is critical for Tensor Omics. Understanding Fortran’s passing conventions allows maximization of speed, minimization of hidden temporaries, and preservation of F42 computational standards.

2 Critical Practices in TOX Interfacing and Control Flow

This section consolidates three key implementation standards for TOX and F42 compliance: correct R–Fortran interfacing, prohibition of `GOTO`, and manual memory cleanup.

2.1 Why `bind(C)` Is Dangerous for R Interfacing

Using `bind(C)` in Fortran changes the function’s symbol name and calling convention to conform to the C ABI. This is incompatible with R’s native `.Fortran()` interface, which relies on legacy Fortran symbol conventions (e.g., appending underscores, pass-by-address semantics).

Problems Caused by `bind(C)` in R Packages

- **Breaks Symbol Lookup:** R expects mangled names like `mysub_`, which `bind(C)` suppresses.
- **Incorrect Argument Handling:** Changes to calling convention may result in wrong memory alignment or crashes.
- **Unnecessary:** R handles native Fortran linking automatically during package build.

Correct Way to Interface R with Fortran

1. Write your subroutines in pure Fortran *without* `bind(C)`.
2. Place Fortran files in `src/` directory of the package.
3. Register your compiled code using `useDynLib(pkgname)` in `NAMESPACE`.
4. Call Fortran code from R via `.Fortran()`:

```
result <- .Fortran("mysub",
  as.double(x), as.double(y), result = double(1))$result
```

Minimal Example of Package Setup

- **DESCRIPTION:** no special flags needed.
- **NAMESPACE:** useDynLib(mytoxpkg)
- **src/mysub.f90:**

```
subroutine mysub(x, y, result)
  real(8) :: x, y, result
  result = x + y
end subroutine
```

2.2 Why GOTO Is Forbidden in F42 and TOX

GOTO is a low-level control structure that violates the principles of structured, readable, and verifiable code. While allowed by Fortran, its use is incompatible with both modern software engineering and WebAssembly (WASM) compilation.

Reasons for Prohibition

- **Non-Structured Control Flow:** Makes code hard to reason about and maintain.
- **Incompatible With WASM:** WebAssembly requires structured control graphs.
- **Opaque for Debugging:** Introduces implicit jump logic that hinders traceability.
- **Fails F42 Goals:** Violates statelessness, modularity, and verifiability.

2.3 How to Replace GOTO Correctly

1. Use exit to break out of a loop

```
do i = 1, n
  if (array(i) < 0.0) then
    exit
  end if
end do
```

2. Use cycle to skip to next iteration

```
do i = 1, n
  if (.not. valid(i)) cycle
  call process(i)
end do
```

3. Use select case for structured branching

```
select case(code)
case(1)
  call handle_case1()
case(2)
  call handle_case2()
```

```

case default
  call handle_default()
end select

```

2.4 Memory Cleanup: Manual Destructors

Since Fortran has no `try/finally` construct and F42 disallows `final ::` destructors due to their opacity, all memory cleanup must be performed manually.

Pattern: Explicit Cleanup Subroutine

```

subroutine deallocate_datatable(dt)
  type(DataTable), intent(inout) :: dt

  if (allocated(dt%real_array)) deallocate(dt%real_array)
  if (allocated(dt%char_array)) deallocate(dt%char_array)
  if (allocated(dt%int_array)) deallocate(dt%int_array)
  if (allocated(dt%column_names)) deallocate(dt%column_names)
end subroutine

```

Pattern: Simulated try-finally via logical block

```

logical :: success
success = .false.

call allocate_datatable(dt, ierr)
if (ierr == 0) then
  call compute(dt)
  success = .true.
end if

call deallocate_datatable(dt)

```

Conclusion

All R-Fortran interfacing, control flow, and memory management in TOX must follow F42 principles:

- Never use `bind(C)` unless wrapping via `.Call()` (not `.Fortran()`).
- Never use `GOTO` — always use structured constructs.
- Always provide manual cleanup subroutines for types containing allocatables.

These principles ensure safety, readability, WASM compatibility, and long-term maintainability.

3 Unified Parallel Loop Snippet

To streamline parallel execution across different computing environments, we define a unified Fortran snippet that automatically generates precompiler code enabling compilation to one of three execution modes: **Coarrays**, **OpenMP**, or **serial fallback**. This is achieved using preprocessor directives that adapt the loop structure based on compiler flags. The loop body itself remains unchanged, ensuring that the parallelization mechanism is orthogonal to the logic being executed.

Below is the result of expanding the `f42:do-parallel` snippet, which selects the appropriate backend at compile time:

```

! -----
! START OF PARALLEL LOOP
! -----
# ifdef USE_COARRAY
start_i = (this_image() - 1) * N / num_images() + 1
end_i   = min(this_image() * N / num_images(), N)

do i = start_i, end_i
# endif

# ifdef USE_OPENMP
!$omp parallel do private(i)
do i = 1, N
# endif

#if !defined(USE_COARRAY) && !defined(USE_OPENMP)
do i = 1, N
#endif

! BODY OF THE PARALLEL LOOP
! -----

! -----
! END BODY OF THE PARALLEL LOOP

end do ! END OF DO-LOOP

# ifdef USE_COARRAY
sync all
# endif

# ifdef USE_OPENMP
!$OMP END PARALLEL
# endif

! -----
! END OF PARALLEL LOOP
! -----

```

Compile with:

- `-DUSE_OPENMP` and `-fopenmp` for OpenMP
- `-DUSE_COARRAY` for Coarrays (e.g. with `-fcoarray=single` for shared memory, or `-fcoarray=lib` for distributed)

3.1 Guideline: Use `f42:do-parallel` only in “Root Loops”

To ensure clarity, efficiency, and robustness of parallel execution in Tensor Omics, parallel loops using the `f42:do-parallel` snippet should only be used at the *root level* of computational routines—that is, in the outermost loops directly executed within a parallelized function or subroutine. These *root-loops* are not nested inside other loops or deeply obscured by function calls.

This restriction avoids unnecessary overhead, data locality issues, and potential thread contention that can arise when parallelization is applied within inner or nested computation layers. Keeping parallelization

explicit and confined to root-loops ensures predictable control over memory access and parallel execution, aligning with the minimalist and efficient design principles of F42. Developers are encouraged to refactor inner computations into pure, stateless subroutines and apply parallelization only where it governs the outermost iteration logic.

4 Fortran to other programming languages Interface

Ideally, we build our Tensor Omics and F42 code so that it can be included and called from any other programming language. This comprises R, of course, but also Python, Lua, Julia, Matlab, etc. We can easily achieve this by offering two interface functions for every Tensor Omics or F42 function that we implement. Importantly, we should implement such pure Fortran functions in Fortran **modules**. Then add another function of the same name with appended “_R” to expose the module function to R and a **bind(C)** with **use_iso_c_binding** and naming the function with appended “_C”. See the below example for clarification.

```
! High-performance module code
module tensor_omics
contains
  subroutine project_vector_core(v, n)
    real(8), intent(inout) :: v(n)
    integer, intent(in) :: n
    ! core vector projection logic
  end subroutine
end module

! R interface
subroutine project_vector_R(v, n)
  real(8) :: v(*)
  integer :: n
  use tensor_omics
  call project_vector_core(v, n)
end subroutine

! C / WASM / Python interface
subroutine project_vector_C(v, n) bind(C, name="project_vector_C")
  use iso_c_binding
  real(c_double) :: v(*)
  integer(c_int) :: n
  use tensor_omics
  call project_vector_core(v, n)
end subroutine
```

4.1 Safe and Efficient Interface Design for bind(C) in Tensor Omics

To expose Tensor Omics routines safely and portably, we use minimal **bind(C)** wrappers around internal Fortran procedures. Each wrapper is designed for ABI stability, no-copy interop, and zero semantic loss.

4.1.1 Keep bind(C) wrappers minimal

Wrap only, no computation.

```
subroutine project_vector_C(x, n) bind(C, name="project_vector_C")
  use iso_c_binding
  real(c_double) :: x(*)
  integer(c_int) :: n
  call project_vector_core(x, n)
```

```
end subroutine
```

4.1.2 Use iso_c_binding types

Avoid compiler-specific kinds.

```
real(c_double) :: x(*)
integer(c_int) :: n
```

4.1.3 Accept arguments as C-style pointers

Do not use assumed-shape or allocatable arrays.

```
real(c_double) :: x(*)      ! OK
real(c_double), dimension(:) :: x ! Not allowed
```

4.1.4 Avoid optional arguments and intent propagation

Declare arguments explicitly, without intent or optional.

```
real(c_double) :: x(*)
! No intent or optional keyword
```

4.1.5 Pass all arguments by reference

Do not use value; pass pointers from caller (Python, Lua, etc.).

```
real(c_double) :: x(*)
integer(c_int) :: n
```

4.1.6 Preserve SIMD and optimization pathways

Call computational logic in a separate routine that uses full Fortran features.

```
subroutine project_vector_core(x, n)
  real(8), intent(inout) :: x(n)
  integer, intent(in) :: n
  ! fast loop, vectorizable
end subroutine
```

4.1.7 Use stable symbol names

Avoid compiler-dependent mangling by specifying names explicitly.

```
subroutine project_vector_C(x, n) bind(C, name="project_vector_C")
```

4.1.8 Ensure ABI compatibility with all consumers

Expose only basic types: scalars and flat arrays.

```
real(c_double) :: vec(*)
integer(c_int) :: n
! No derived types, assumed-shape arrays, or allocatables
```

4.1.9 Verify type equivalence when calling core Fortran routines

Make sure internal types match `iso_c_binding` externally.

```
use iso_c_binding
real(8) :: x(n)
! Ensure real(8) == real(c_double) on your compiler
```

4.1.10 Avoid aliasing in calls to `bind(C)` routines

Do not pass the same variable twice under different names.

```
! DON'T - Unsafe aliasing: same memory twice
call project_vector_C(x, x, n)
```

```
! OK - Pass unique memory regions only
call project_vector_C(x, y, n)
```

This structure ensures maximum compatibility and reproducibility across calling environments without compromising Fortran-native performance.

4.1.11 Flattening Multidimensional Arrays for `bind(C)` Interfaces

Multidimensional arrays such as 2D expression matrices or tensor fields cannot be passed directly through `bind(C)` interfaces, since assumed-shape arrays (e.g., `(:,:)` or `(:,:,:)` types) require Fortran array descriptors that are not compatible with C.

To expose such routines safely, we flatten the array into a 1D buffer and pass shape metadata separately. The internal Fortran logic reshapes the buffer without copying.

C-compatible wrapper:

```
subroutine process_matrix_C(flat, nrow, ncol) bind(C, name="process_matrix_C")
  use iso_c_binding
  real(c_double), intent(inout) :: flat(*)
  integer(c_int), value :: nrow, ncol
  call process_matrix(reshape(flat, [nrow, ncol]))
end subroutine
```

Internal Fortran routine:

```
subroutine process_matrix(mat)
  real(8), intent(inout) :: mat(:, :)
  integer :: i, j
  do j = 1, size(mat, 2)
    do i = 1, size(mat, 1)
      mat(i, j) = mat(i, j) * 2.0
    end do
  end do
end subroutine
```

Caller responsibility: The external caller (e.g., from C, Python, Lua) must allocate a 1D array of size `nrow * ncol` and ensure it is stored in Fortran-style column-major order, or reshaped accordingly. This pattern allows Tensor Omics to retain full multidimensional structure internally while offering safe and portable interfaces to external consumers.

5 Fortran Array Notation Cheat-Sheet

This section summarizes the three common ways to declare array arguments in Fortran procedures, with a focus on their use, safety, and suitability in modern code.

5.1 1. Assumed-Size Arrays (*)

```
subroutine example(a, n)
  real(8), intent(in) :: a(*)
  integer, intent(in) :: n
  integer :: i
  do i = 1, n
    print *, a(i)
  end do
end subroutine
```

Meaning: `a(*)` is an assumed-size array. The subroutine does not know the array bounds and must be told explicitly via a separate argument (e.g., `n`). This is the lowest-level array form and maps closely to a raw pointer in C.

Limitations:

- No bounds checking
- `size(a)` and `lbound(a)` are illegal
- Cannot use array operations like `matmul` or `sum`
- Unsafe for slicing (`a(2:5)` is undefined)

Use case: C interoperability, legacy code, low-level memory kernels.

5.2 2. Assumed-Shape Arrays (:)

```
subroutine example(a)
  real(8), intent(in) :: a(:)
  integer :: i
  do i = 1, size(a)
    print *, a(i)
  end do
end subroutine
```

Meaning: `a(:)` is an assumed-shape array. The compiler receives size and bounds information at runtime. Requires an explicit interface (typically via a module or `interface` block).

Advantages:

- Safe: `size(a)` is available
- Supports slicing, broadcasting, and array intrinsics
- Can use `matmul`, `dot_product`, etc.

Use case: Recommended default for most modern Fortran code.

5.3 3. Multidimensional Arrays (:,:) and Higher

```
subroutine process_matrix(A)
  real(8), intent(in) :: A(:, :)
  print *, size(A, 1), size(A, 2)
end subroutine
```

Meaning: Each colon represents a dimension of unknown size. Like 1D assumed-shape arrays, this requires an explicit interface and passes shape info at runtime.

Features:

- Full array operations: `transpose`, `matmul`, etc.
- Safer and clearer semantics for tensors and matrices

Use case: Tensor Omics vector spaces, image/matrix processing, linear algebra routines.

Summary Table

Notation	Bounds Known?	Safe?	Use Case
<code>(*)</code>	No	No	C interop, low-level
<code>(:)</code>	Yes	Yes	Modern Fortran arrays
<code>(:,:)</code>	Yes	Yes	Matrices, tensors

For all assumed-shape forms `((:), (:,:), (:,:))`, ensure the subroutine has an explicit interface or is in a module.